Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Huffman kodlamanın tekrarı ve iki farklı ağaç (ekrandan, 00:10:00 ve 00:20:00)
  - Huffman, Shannon-Fano'nun üzerine geliştirilmiş, olasılığa dayalı bir yöntemdir. Tek başına ya da başka bir yöntemin içinde ara adım olarak kullanılır.
  - Olasılıklar küçükten büyüğe sıralanır, en küçük iki değer toplanır ve toplam sıralamayı bozmayacak yere konur; tek değer (1,0) kalana kadar sürer.
  - Toplam, kendisine eşit bir değerin önüne ya da arkasına konabilir. Olasılıklar $0{,}4; 0{,}2; 0{,}2; 0{,}1; 0{,}1$ için iki yol iki farklı kod kümesi verir: `0, 10, 111, 1101, 1100` ve `11, 01, 00, 101, 100`. İkisinde de $S = 2{,}2$ bit/sembol ve entropi aynıdır.
- Kod uzunluklarının varyansı (ekrandan, 00:30:00)
  - Varyans $\sum_i P_i(l_i - S)^2$ ile hesaplanır. Birinci ağaçta 1,36, ikinci ağaçta 0,16 çıkar.
  - Sıkıştırılmış veri dosyaya yazılacaksa varyansın önemi yoktur. Veri bir iletişim hattından gönderilecekse küçük varyanslı kod seçilir.
  - Gönderme hızı ile kod üretme/çözme hızı aynı değilse beklemeyi önlemek için buffer kullanılır. Varyans büyükse buffer büyük, sıfıra yakınsa küçük seçilebilir.
  - Slayttaki kural: a_i sembolünün kod uzunluğu $\lceil -\log_2 P_i \rceil$ ile eşit ya da ondan küçüktür. $P = 0{,}4$ için $\lceil 1{,}32 \rceil = 2$ bit; sembol 2 ya da 1 bitle kodlanır.
  - Ortalama mesaj uzunluğu kısa yoldan da bulunur: ağaçtaki ara toplamların (iç düğümlerin) toplamı $0{,}2 + 0{,}4 + 0{,}6 + 1{,}0 = 2{,}2$ verir.
- Sıklık değerleriyle Huffman ve genişletilmiş dizi (ekrandan, 00:33:30 ve 00:40:00)
  - Örnek: `a, e, k, l, m, z` sembolleri sırasıyla 25, 14, 8, 7, 4, 2 adet. Huffman olasılıkla da sıklıkla da yapılabilir.
  - Sıklıklar küçükten büyüğe sıralanır: `2 4 7 8 14 25`. Soldan iki değer toplanır, toplam negatif işaretle (ara düğüm olduğunu göstermek için) sıralamayı bozmayacak yere konur. Toplarken işaret dikkate alınmaz.
  - Sonuç dizi: `2 4 -6 7 8 -13 14 -21 25 -35 -60`. n sembol için genişletilmiş dizi $2n - 1$ elemanlıdır ($n = 6$ için 11).
  - Ağaç: son eleman (-60) kök olur; sondan geriye doğru ikişer okunur, ilk okunan sol dala, ikincisi sağ dala konur. Negatifler ara düğüm, pozitifler yapraktır.
  - Sol dallara 0, sağ dallara 1 verilir ve kodlar kökten yaprağa okunur: a=`1`, e=`01`, k=`001`, l=`0000`, m=`00010`, z=`00011` (ekrandan, 00:50:00).
  - Kodlar aşağıdan yukarı okunsaydı (`1, 10, 100, 0000, 01000, 11000`) prefix olmazdı.
- Başlık (header) bilgisi ve ağacın gönderilmesi (ekrandan, 01:00:00 ve 01:10:00)
  - Açıcı tarafın kodları bilmesi için sıkıştırılmış dosyanın başına bir header eklenir. Sembol sayısı, semboller ve sıklıkları gönderilebilir; ancak sıklık değerlerinin kaç bayt tutacağı belli olmadığı için header büyür ve bazen sıkıştırılmış veri kadar olabilir.
  - Ağaç, Left-Node-Right notasyonuyla dal etiketleri dizisi olarak gönderilebilir. Örnek ağaç için bit dizisi `0000101111`, yaprak sırası `l m z k e a`. Header'a kaç bit okunacağı (10) ve semboller de yazılır.
  - Açıcı tarafta 0 sola inmek demektir. 1 geldiğinde bulunulan yer yapraktır; sıradaki sembol oraya yazılır ve sağ dalı işlenmemiş ilk üst düğümün sağına geçilir. Huffman ağacında sol dal olmadan sağ dal olmaz.
- Genişletilmiş diziden doğrudan kod üretme (ekrandan, 01:20:00, 01:30:00 ve 01:33:45)
  - Kökün indisi 0'dır. Bir düğümün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$ indisini alır. Dizi sondan başa taranır, her negatif eleman için iki çocuk indisi sıraya yazılır.
  - Sonuç: -60→0, -35→1, 25→2, -21→3, 14→4, -13→7, 8→8, 7→15, -6→16, 4→33, 2→34.

  - Kod için ters işlem yapılır: indis çiftse `1` yazılır ve $i = (i-2)/2$, tekse `0` yazılır ve $i = (i-1)/2$; indis 0 olana kadar sürer. Bulunan bitler sağdan sola yazılır.
- Statik, yarı dinamik ve dinamik Huffman (ekrandan, 01:40:00)
  - Statik: bir derlemdeki sıklıklardan kodlar bir kez çıkarılır, her dokümana aynı kodlar uygulanır.
  - Yarı dinamik: doküman iki geçişte işlenir; önce sıklıklar çıkarılır, sonra Huffman kodları üretilir. Statikten daha iyi sıkıştırır.
  - Dinamik: encoder ve decoder senkronize çalışıp aynı anda aynı ağacı kurar. Bir sembolün kodu doküman içinde değişebilir. Hızın önemli olduğu, iletim hattı üzerinden gönderilen verilerde kullanılır; varyansı küçük ağaç tercih edilir.
  - Başlangıçta ağaçta yalnız escape kodu vardır. Yeni bir sembol önce escape koduyla, sonra kodlanmadan (ASCII) gönderilir ve ağaca eklenir. Bilinen sembol için kodu gönderilir ve sıklığı bir artırılır. Escape kodunun sıklığı hep 1'dir.
  - Ağaç değişimi örneği (ekrandan, 01:50:00): A(1), B(2), C(2), D(2), E(9) ile başlayan ağaçta A geldikçe toplam 16, 17, 18, 19 olur; A'nın sıklığı 3 olunca ve 5 olunca ağaç yeniden düzenlenir.
  - Sıklık sayaçları işaretsiz 16 bitte tutulur; 65.535'e ulaşınca bütün sıklıklar tamsayı bölmesiyle ikiye bölünür. Ağaç derinliği kod alanını aşarsa alan genişletilir ya da bağlı liste kullanılır.
- Huffman'ın farklı bir versiyonu (ekrandan, 02:00:00)
  - Basit ama daha az etkili bir yöntemdir. Eşit olasılıklı sembollere uygundur; olasılıklar eşit değilse etkili değildir.
  - $n \times 3$ tablo: sembol, sayaç (count) ve kod. Sayaç değiştikçe satırlar yer değiştirir, kod kolonu sabit kalır. Sayaç taşarsa değerler yarıya bölünür.
- Canonical Huffman kodlama (ekrandan, 02:10:00 ve 02:20:00)
  - Geniş alfabeli dosyalarda daha küçük başlık sağlar, hızlı kod dönüşümüne sahiptir, Huffman'ın atladığı prefix kodların oluşturulmasına izin verir. Hoca ses ve arşiv dosyalarında kullanıldığını söyledi.
  - Huffman kod uzunlukları büyükten küçüğe sıralanır. En uzun uzunluk L_{max}, bu uzunlukta k_1 kod varsa 0'dan başlayan k_1 ardışık binary sayı yazılır. Sonraki uzunluk için önceki grubun son kodunun soldaki bitleri alınıp 1 eklenerek devam edilir.
  - Örnek: uzunluklar (5, 5, 5, 5, 3, 2, 2, 2) → `00000, 00001, 00010, 00011, 001, 01, 10, 11`.
- MNP5 (ekrandan, 02:26:50, 02:30:00, 02:34:00 ve 02:37:30)
  - Microcom'un modemler için geliştirdiği iki aşamalı yöntem: run-length encoding ve adaptive frequency encoding. MNP5 ve MNP7 sürümleri vardır; en çok MNP5 tercih edilir.
  - RLE aşaması: üç ya da daha fazla tekrar eden sembolün üçü yazılır, dördüncü bayta tekrar sayısı yazılır.
  - İkinci aşama: 256 satırlık tablo; her satır bir bayt değeri, sıklık (başlangıçta 0) ve 3 bitlik header ile başlayan değişken uzunluklu bir token içerir. Tokenlar 4 ile 11 bit arasındadır.
  - Okunan baytın token'ı yazılır, sıklığı bir artırılır; sıklığı büyüyen bayt üstteki daha kısa token'ı alır (yalnız tokenlar yer değiştirir, pointerlarla yapılabilir).
  - RLE'nin tekrar sayısı da tablodan kodlanır ama sıklığı artırılmaz. Örnek `52 52 52 6`: 52 için 53. satırın token'ı, 6 için 7. satırın token'ı yazılır.
  - Sıklık 8 bitte tutulur, dolarsa bütün sıklıklar ikiye bölünür. Sona 11 tane 1'den oluşan flush token eklenir, gerekirse 8'e tamamlanır.
- MNP7 (ekrandan, 02:44:30)
  - MNP5'ten daha karmaşık, iki boyutlu bir sürümdür; RLE ve adaptif Huffman benzeri kodlama birinci dereceden Markov modeline dayanır.
  - Token, şimdiki karakter ile ondan önceki karaktere göre seçilir. Her önceki karakter için 256 satırlık bir sıklık tablosu vardır; kodlar ayrı tabloda tutulur ve yer değiştirmez.

Hocanın Özellikle Vurguladığı Kısımlar

- Varyansın neden önemli olduğu
  - Entropi, ortalama mesaj uzunluğu ve artıklık iki ağaçta aynıdır. Fark yalnız iletim

hattında ortaya çıkar: uzun ve kısa kodlar karışıksa alıcı tarafta bekleme olur ve daha büyük bir buffer gerekir. Bu yüzden dinamik (adaptif) Huffman'da küçük varyanslı ağaç seçilir.
- Ağaç her zaman aynı kuralla kurulmalı
  - İlk okunan sol dala, ikinci okunan sağ dala konacaksa bu hep böyle yapılmalıdır. Kural keyfe göre değişirse çıkan kodlar prefix olmaz.
- Kodlar kökten yaprağa okunur
  - Huffman ağacı aşağıdan yukarı kurulur ama kodlar yukarıdan aşağı okunur. Aksi halde kodlar prefix olmaz.
- Huffman karmaşık değildir
  - Elde iki dizi (genişletilmiş dizi ve indisler) ile "ikiye böl, bir ya da iki çıkar" işlemleriyle bütün kodlar çıkarılır. Header'da ağaç gönderilirse başlık da kısalır.
- Huffman'ın başarısı
  - Hoca, doğal dilde yazılmış bir metinde başarının %65'e kadar çıktığını, %70'in çok nadir olduğunu, dinamik Huffman'la daha yükseğe çıkılabileceğini söyledi.
- Algoritma varyantları
  - Canonical Huffman çok kullanılan bir yöntem değildir. Hoca bunu, araştırmacıların bir algoritmayı değiştirip belirli veri için daha iyi sonuç veren yeni sürümler ürettiğini göstermek için anlattığını söyledi.

Kısa Tekrar Notları

- Huffman: küçükten büyüğe sırala, en küçük ikiyi topla, sıralamayı bozmadan yerleştir.
- Varyans: $\sum P_i(l_i - S)^2$. Örnekte 1,36 ve 0,16. İletim hattında küçük varyanslı kod seçilir.
- S = ara toplamların toplamı ($0{,}2 + 0{,}4 + 0{,}6 + 1 = 2{,}2$).
- Genişletilmiş dizi $2n - 1$ eleman; ara toplamlar negatif yazılır; son eleman kök.
- Sol dal 0, sağ dal 1; kod kökten yaprağa okunur.
- İndisler: sol $2i+1$, sağ $2i+2$. Kod: çift indis → `1`, $i = (i-2)/2$; tek indis → `0`, $i = (i-1)/2$; bitler sağdan sola.
- Header'da ağaç: Left-Node-Right dal dizisi (`0000101111`) + yaprak sırası (`lmzkea`) + bit sayısı.
- Statik (derlemden hazır kod), yarı dinamik (iki geçiş), dinamik (senkron ağaç, escape kodu).
- Dinamik Huffman'da sayaç taşarsa bütün sıklıklar ikiye bölünür.
- Canonical Huffman: en uzun grup 0'dan ardışık; sonraki grup önceki son kodun soldaki bitleri + 1.
- MNP5 = RLE (3 tekrar + sayı) + 256 satırlık adaptif token tablosu; flush token 11 tane 1.
- MNP7 = iki boyutlu, birinci dereceden Markov; önceki karaktere göre tablo.

Detaylı Açıklamalar

- İki Huffman ağacı (ekrandan, 00:15:40 ve 00:20:00)
  - Olasılıklar $a_1 = 0{,}4$, $a_2 = 0{,}2$, $a_3 = 0{,}2$, $a_4 = 0{,}1$, $a_5 = 0{,}1$.
  - Birinci yol: $0{,}1 + 0{,}1 = 0{,}2$ eşit değerlerin önüne konur; sonra $0{,}2 + 0{,}2 = 0{,}4$ yine öne; $0{,}4 + 0{,}2 = 0{,}6$; $0{,}6 + 0{,}4 = 1$. Kodlar `0, 10, 111, 1101, 1100`; $S = 0{,}4 \cdot 1 + 0{,}2 \cdot 2 + 0{,}2 \cdot 3 + 0{,}1 \cdot 4 + 0{,}1 \cdot 4 = 2{,}2$.
  - İkinci yol: toplam eşit değerlerin arkasına konur. Kodlar `11, 01, 00, 101, 100`; $S = 0{,}4 \cdot 2 + 0{,}2 \cdot 2 + 0{,}2 \cdot 2 + 0{,}1 \cdot 3 + 0{,}1 \cdot 3 = 2{,}2$.
  - Entropi derste sayısal hesaplanmadı; python ile $H \approx 2{,}12$ bit, $R \approx 0{,}08$.
- Varyans hesabı (ekrandan, 00:30:00)
  - Birinci ağaç: $0{,}4(1-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(3-2{,}2)^2 + 0{,}1(4-2{,}2)^2 + 0{,}1(4-2{,}2)^2 = 1{,}36$.
  - İkinci ağaç: $0{,}4(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}1(3-2{,}2)^2 + 0{,}1(3-2{,}2)^2 = 0{,}16$ (ikisi de python ile doğrulandı).
  - Hoca sezgi için `1 1 2 1 1` (varyansı küçük) ve `3 1 7 4` (varyansı büyük) dizilerini karşılaştırdı.
  - **Ders sonrası not:** Hoca varyansı "ortalamadan farkların karelerinin karekökü" diye tanımladı; karekök alınınca standart sapma elde edilir. Varyans, farkların karelerinin

(burada olasılıkla ağırlıklı) ortalamasıdır; slayttaki hesap da böyledir.
  - Hoca $\lceil \cdot \rceil$ işaretinin yönünden emin olamadı; aradan sonra bir öğrenci $\log_2 0{,}4 \approx -1{,}32$ olduğunu, yani $\lceil 1{,}32 \rceil = 2$ sonucunun doğru olduğunu söyledi (python ile doğrulandı).
- Genişletilmiş dizinin kurulması (ekrandan, 00:40:00)

```
başlangıç:  2  4  7  8  14  25
2+4=6   ->  2  4  -6  7  8  14  25
6+7=13  ->  2  4  -6  7  8  -13  14  25
8+13=21 ->  2  4  -6  7  8  -13  14  -21  25
14+21=35 -> 2  4  -6  7  8  -13  14  -21  25  -35
25+35=60 -> 2  4  -6  7  8  -13  14  -21  25  -35  -60
```

- Slayt: en sondaki eleman kök olur; bir sonraki ($2n-2$. konum) sol dala, ($2n-3$. konum) sağ dala yerleşir. -60 kök, -35 sol, 25 sağ; -35'in altına -21 (sol) ve 14 (sağ); -21'in altına -13 ve 8; -13'ün altına 7 ve -6; -6'nın altına 4 ve 2.
- Bu kodlarla toplam $25 \cdot 1 + 14 \cdot 2 + 8 \cdot 3 + 7 \cdot 4 + 4 \cdot 5 + 2 \cdot 5 = 135$ bit, sembol başına $135/60 = 2{,}25$ bit. Derste hesaplanmadı; python ile. Ara toplamların toplamı da $6 + 13 + 21 + 35 + 60 = 135$ verir.
- Ağacı Left-Node-Right notasyonuyla gönderme (ekrandan, 01:00:00)
  - Okuma: kökten sola `0`, sola `0`, sola `0`, sola `0` → yaprak l; üst düğümden sağa `1`; sola `0` → m; sağa `1` → z; üst düğümden sağa `1` → k; sağa `1` → e; kökten sağa `1` → a. Dizi `0000101111`, yaprak sırası `lmzkea`.
  - Header'ın bir baytına okunacak bit sayısı (10) yazılır, ardından bu bit dizisi ve semboller gelir (her sembol bir bayt).
  - Çözme: açıcı her 0 için bir sol dal açar. 1 geldiğinde bulunduğu yer yapraktır, sıradaki sembolü yazar; sonra yukarı çıkıp sağ dalı açılmamış ilk düğümün sağına iner. Sol dal yokken sağ dal olamayacağı için, bir sağ dala indikten hemen sonra yine 1 gelirse o düğüm de yapraktır.
  - Ağaç kurulduktan sonra sıkıştırılmış veri bit bit okunur: `0`, `00`, `000` kod değil; `0000` = l, vb.
- Diziden kod üretimi (ekrandan, 01:30:00 ve 01:33:45)

```
While indis <> 0 do
{
  if indis çift then
    { print "1"
      indis = (indis-2)/2 }
  else
    { print "0"
      indis = (indis-1)/2 }
}
```

- 25 (indis 2): çift → `1`, $(2-2)/2 = 0$. Kod `1`.
- 14 (indis 4): çift → `1`, $i = 1$; tek → `0`, $i = 0$. Sağdan sola: `01`.
- 8 (indis 8): `1`, $i = 3$; `0`, $i = 1$; `0`, $i = 0$ → `001`.
- 7 (indis 15): `0`, $i = 7$; `0`, $i = 3$; `0`, $i = 1$; `0`, $i = 0$ → `0000`.
- 4 (indis 33) → `00010`; 2 (indis 34): `1`, $i = 16$; `1`, $i = 7$; `0`, `0`, `0` → `00011`. Bütün kodlar ağaçtan okunanlarla aynı (python ile doğrulandı).
- Bitler sağdan sola yazılır, çünkü işlem yapraktan köke doğru ilerler, kod ise kökten yaprağa okunur.
- Dinamik Huffman'da escape kodu (ekrandan, 01:42:40)
  - Başta iki tarafın ağacında yalnız escape vardır, kodu `0`'dır. İlk `b` için önce `0`, sonra kodlanmamış `b` gönderilir; iki taraf da `b`'yi ağaca ekler. İkinci `b` için yalnız `b`'nin kodu gönderilir, iki tarafta da sıklık 2 olur ve ağaç aynı biçimde yeniden düzenlenir.
  - **Soru:** Bir dokümanda dinamik Huffman ile elde edilen son ağaç, sonraki dokümanlar için referans olarak kullanılamaz mı? **Cevap:** Bu statik kodlama olur ve başarıyı düşürür; başka dokümanda sıklıklar değişeceği için kodlar da değişir. Yaklaşık bir sonuç verebilir ama hangi dokümanda ne kadar iyi olacağı bilinemez.
- Huffman'ın farklı versiyonu (ekrandan, 02:00:00)
  - Başlangıç tablosu a1, a2, a3, a4 (sayaç 0, kodlar `0, 10, 110, 111`). a2 gelir: sayacı

1, en üste çıkar. a4 gelir: sayacı 1, a2'nin altına çıkar. Bir a4 daha gelir: sayacı 2, en üste çıkar. Son tablo a4 `0`, a2 `10`, a3 `110`, a1 `111`.

- Canonical Huffman örneği (ekrandan, 02:20:00)
  - $L_{max} = 5$, $k_1 = 4$: `00000, 00001, 00010, 00011`.
  - $L = 3$, $k_2 = 1$: son kod `00011`'in soldaki 3 biti `000`, $+1 \rightarrow$ `001`.
  - $L = 2$, $k_3 = 3$: `001`'in soldaki 2 biti `00`, $+1 \rightarrow$ `01`, sonra `10`, `11`.
  - **Ders sonrası not:** Hoca bir yerde "soldaki 3 biti at, kalanı al" dedi, sonra düzeltti. Slayttaki işlem soldaki L biti almak (sağdaki fazla bitleri atmak) ve 1 eklemektir.
- MNP5 token tablosu (ekrandan, 02:30:00 ve 02:39:50)
  - Bayt 0-1: `000|0`, `000|1`; 2-3: `001|0`, `001|1`; 4-7: `010|00` ... `010|11`; 8-15: `011|000` ... `011|111`; 32-34: `101|00000` ...; 247-254: `111|1110111` ... `111|1111110`; 255: `111|11111110`.
  - Slayt: üç özel kod dışında 2 tane 1 bitlik, 4 tane 2 bitlik, 8 tane 3 bitlik, 16 tane 4 bitlik, 32 tane 5 bitlik, 64 tane 6 bitlik ve 128 tane 7 bitlik kod vardır. Özel kodlar `000|0`, `000|1` ve `111|11111110`'dur.
  - **Ders sonrası not:** Slayttaki tabloda 16-31 baytlarının header'ı `111` yazılmış (`111|0000` ...). Hoca burasının düzeltilmesi gerektiğini söyledi ("şurası 4 olacak"). Header'lar grup sırasına göre arttığı için 16-31 grubunun header'ı büyük olasılıkla `100` olmalıdır.
  - Örnek `52 52 52 6`: 9 tane 52 demektir. Üç 52'nin her biri için o anki token yazılır ve sıklık artar (token değişebilir); 6 tekrar sayısı olduğu için 7. satırın token'ı yazılır, sıklığı artırılmaz.
- MNP7 (ekrandan, 02:44:30)
  - Örneğin `ab` dizisinde `b` işlenirken önceki karakter `a`'nın tablosuna ve `b`'nin satırına bakılır. Sıklıklar bu tabloda değişir, kodlar ayrı tabloda sabit kalır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.